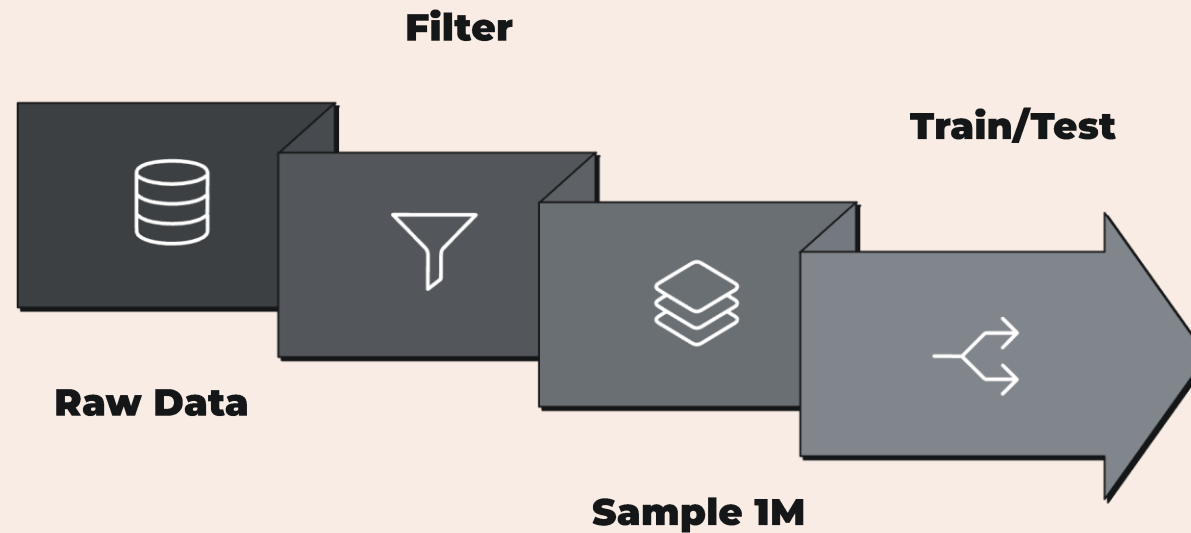


Recommendation Systems for Personalized Content Discovery

By Nishant(24117089) and Vedant Halkude(24112113)

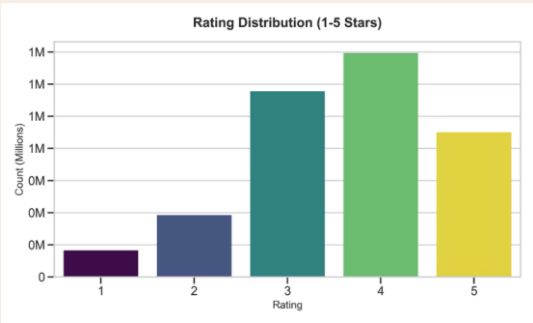
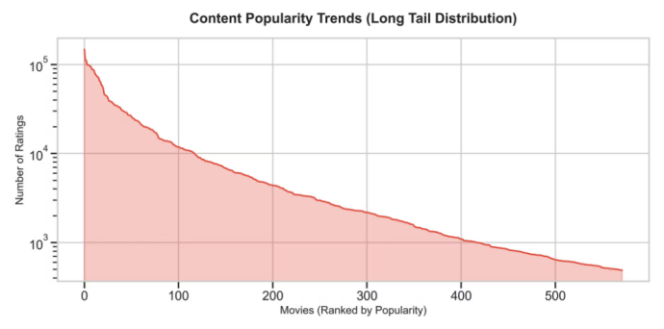
Problem Overview

- The Netflix Prize challenged teams to improve movie recommendation accuracy at scale.
- The core objective is to predict user preferences and surface the most relevant unseen titles.
- Accurate recommendations drive retention, engagement, and long-tail content discovery.
- Business impact: better personalization increases watch-time and helps viewers find content they might otherwise miss.



Problem Overview (continued)

We explored the Netflix Prize dataset to better understand the problem before modeling. It contains **100M+ ratings** from **480K users** across **17K movies**, but the signal is highly uneven: the user-item matrix is **98.8% sparse**, ratings are skewed positive, and activity is concentrated in a small number of power users. This is a classic recommender-systems challenge. The data reflects a long-tail distribution where a few popular titles collect most of the feedback, while many niche movies have little direct evidence. That means a model cannot rely only on popularity – it must infer hidden preferences and personalize recommendations effectively.



Extreme Sparsity

About **98.8%** of the user-item matrix is empty, leaving very few direct overlaps between users. Similarity-based methods have limited shared evidence, so the model must learn from indirect structure.

Positivity Bias

More than **56%** of ratings are **4 or 5 stars**, so the dataset leans heavily toward positive feedback. This can make popular content look stronger than it is unless the recommender is calibrated for diversity.

Long-Tail Distribution

Most movies receive relatively little feedback, while a small set of popular titles attracts disproportionate attention. A strong recommender must still surface relevant niche content that users may love but would not find on their own.

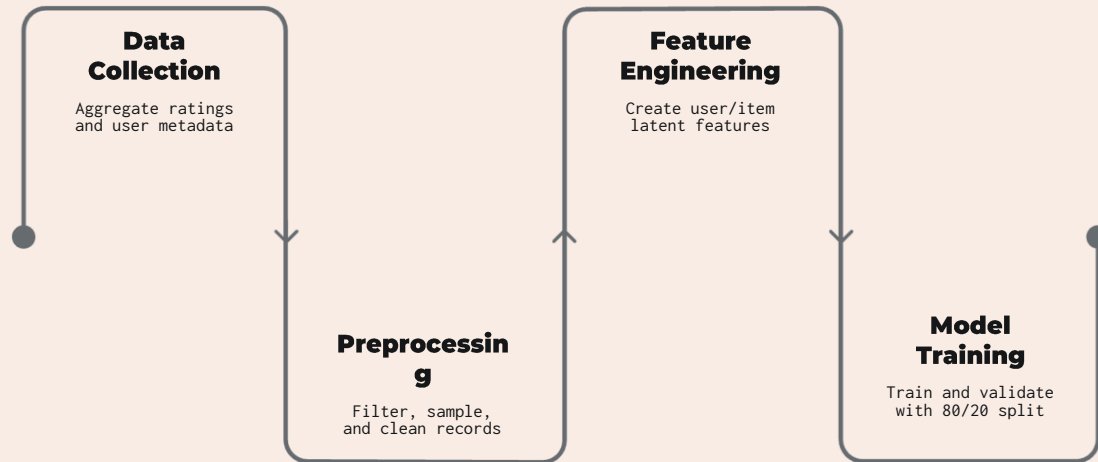
Power Users

The median user has **96 ratings**, while some contributed **more than 17K**. These heavy raters provide valuable signal, but they can also dominate the model if not balanced carefully.

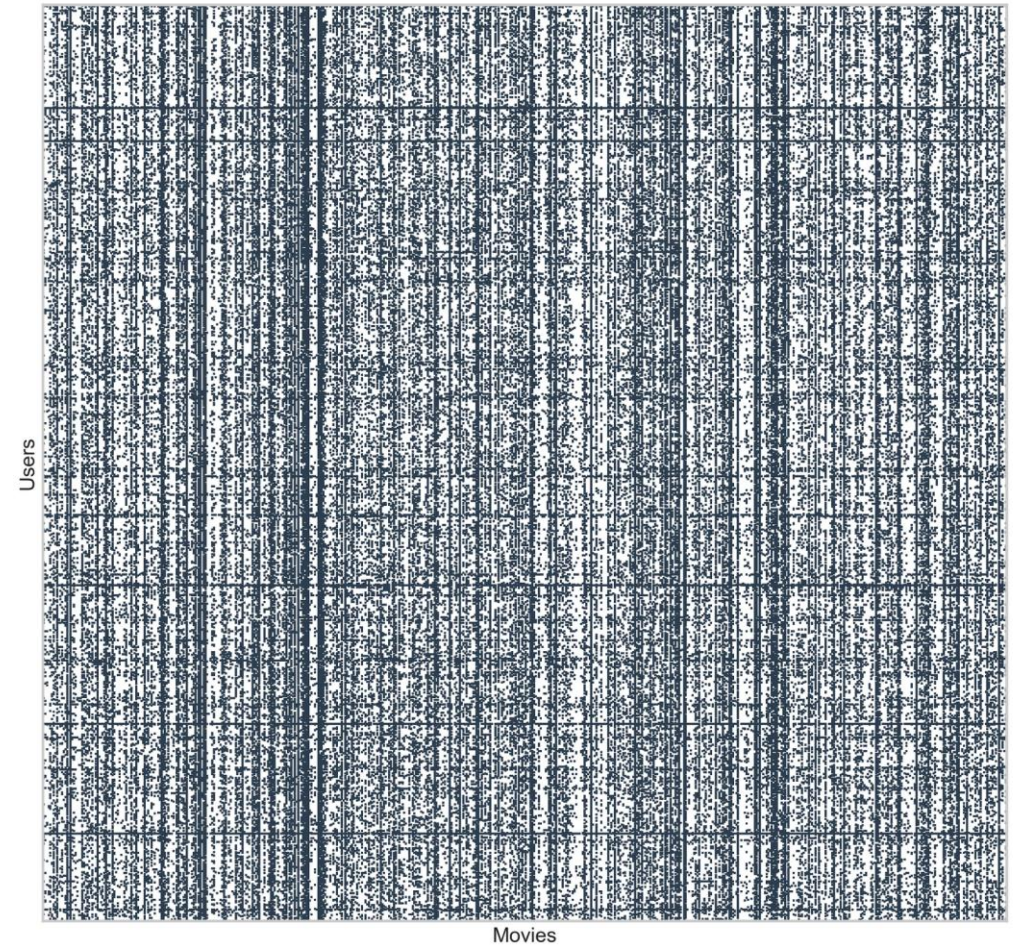
Business Implication

Personalization is essential because the dataset is sparse, biased toward positive ratings, and shaped by uneven participation. Recommenders must uncover hidden preferences, reduce popularity bias, and surface long-tail content to improve engagement and retention.

Approach



Data Sparsity Visualization (Sample 1000x1000)



Approach (continued)

Model Design

We compared two recommender system approaches to understand both performance and behavior on sparse rating data. The first is **Collaborative Filtering** using Item-Based KNN, which recommends items based on observed item-to-item similarity. The second is **Matrix Factorization** using SVD, which learns compact user and item representations from the ratings matrix.

- **Item-Based KNN:** Pearson baseline similarity on observed ratings.
- **SVD:** 50 latent factors learned from user-item interactions.
- **Metrics:** RMSE for raw prediction error and MAP@10 for ranking quality.

RMSE measures how close predicted ratings are to the true ratings, making it a direct measure of prediction error.

MAP@10 evaluates ranking quality by checking whether relevant items appear near the top of the recommendation list, using a relevance threshold of ≥ 3.5 .

Item-Based KNN is the intuitive baseline. It looks for movies that have been rated similarly by the same users, then recommends items “near” the one a user already liked. In practice, this makes the model easy to explain and grounded in real comparisons between items, but it can struggle when the catalog is large and the user-item matrix is sparse.

SVD takes a different path. Instead of comparing items directly, it factorizes the ratings matrix into compact user and item representations with 50 latent factors. Those hidden vectors capture patterns such as taste, genre affinity, and rating tendencies, allowing the model to infer preferences even when explicit overlap is limited.

By putting these models side by side, we aimed to see how much predictive power comes from direct neighborhood similarity versus compressed latent structure, and whether the added abstraction of SVD improves scalability and robustness without sacrificing ranking quality.

1	2		
Collaborative Filtering	Matrix Factorization	Aspect	Takeaway
Item-Based KNN with Pearson baseline similarity	SVD with 50 latent factors, learns user & item vectors	Core idea	Latent models impute sparsity
		Memory	SVD scales better
		Scalability	SVD preferred for large catalogs
		Realtime inference	SVD suits production

Experimental Results

We evaluated Item-Based KNN and SVD on both prediction accuracy and top-10 ranking quality. The results show that **SVD is superior overall**: it achieves lower RMSE, slightly higher MAP@10, and a more practical training and inference profile for production use.

Model	RMSE	MAP@10	Takeaway
SVD	0.9661	0.5953	Best overall performance
KNN	1.1318	0.5895	Weaker accuracy and ranking

Key finding: SVD outperforms KNN on both metrics, with a clear RMSE improvement and a small but meaningful gain in MAP@10.



Lower error with SVD

SVD's RMSE of 0.9661 is substantially better than KNN's 1.1318, meaning its predicted ratings are closer to what users would actually give. This makes SVD more reliable when the goal is accurate preference estimation.



Better top-10 ranking

SVD also edges out KNN on MAP@10, scoring 0.5953 versus 0.5895. That means its ranked recommendation lists are slightly more relevant at the top, which is what users see first.



More efficient and production-ready

KNN requires $O(N^2)$ similarity computation and neighbor lookups, while SVD is trained with **SGD** on compact latent factors. That gives SVD better scalability, faster inference, lower memory usage, and stronger practical usability for production systems.

- SVD is better on both accuracy and ranking quality.
- KNN is simpler conceptually, but expensive to scale.
- SVD is the better choice for efficient, production-ready recommenders.

In short, SVD is superior because it combines stronger predictive performance with a much more scalable computational profile. It is not only more accurate, but also more practical to train, deploy, and serve at scale.

Key Insights

Our evaluation shows that **extreme sparsity** and **skewed user activity** shape both model choice and interpretation. In this setting, **SVD** is the right approach for accuracy and production readiness.



Sparsity defeats memory-based models

With **98.8% sparsity**, user-item overlap is too thin for KNN to find reliable neighbors, so memory-based methods become ineffective. SVD is necessary because it learns latent factors from sparse interactions instead of depending on direct overlap.



Power users skew averages

A small number of users contribute **more than 17k ratings**, which can distort averages and create bias in evaluation. Bias baselines are important to account for these heavy raters before judging model lift.



SVD is stronger for production

SVD offers **fast inference**, **scalability**, and **online learning capability**, making it more practical for real-time recommendation systems and growing datasets.



Future improvements

Next steps include **distributed computing** for larger-scale training, **content-based cold start strategies** for new users and items, and **ensembling with hyperparameter tuning** to further improve recommendation quality.



Overall, the key takeaway is that sparse recommendation data favors latent-factor methods over memory-based approaches, especially when production performance matters.

Recommendation Examples

These examples show a recommender at its best and worst. The strong predictions come from dense genre clusters where latent factors can capture shared preference patterns. The weak predictions appear when a user steps outside their historical bubble or when the model has too little prior signal to rely on.

User ID	Movie	Actual	Predict ed
794999	North by Northwest	5.0	5.00
998461	Aqua Teen Hunger Force: Vol. 1	5.0	5.00
1461435	Lady Chatterley	5.0	1.00
2286441	I Love Lucy: Season 2	1.0	5.00

The first two rows are clean wins: the model matches the true rating exactly because the user and title fit a familiar pattern.

The third and fourth rows show the hard cases. *Lady Chatterley* is a cold-start-style miss: the model does not have enough similar examples, so it underpredicts. *I Love Lucy: Season 2* is the opposite problem: the title looks like a strong fit on paper, but this user rejects it anyway.

- **Explainability:** the strongest recommendations are easiest to justify when they align with shared preference patterns across similar users and movies.
- **Cold-start problem:** users or items with little history are difficult to place, especially when they step outside their historical bubble.
- **Mitigation strategies:** popularity-based baselines, content-based features, and hybrid recommenders can stabilize predictions when collaborative signals are thin.

Bottom line: recommender systems are strongest when behavior is stable and repetitive. They get brittle when users are new, sparse, or unpredictable, so hybrid approaches are often the most practical production choice.

“

Why successes occur

Latent factors work well in dense genre clusters. When the model has many overlapping examples, it can infer shared taste patterns and make a sharp prediction.

”

“

Why failures occur

Cold-start users and out-of-bubble items break the pattern. With weak signals, the model falls back to averages and can miss badly.

”